PRD: SAST and DAST Correlation — Problem, Goals and Requirements

30 October 2026 · Anuj

1. Overview and problem statement

Build a tool that scans an application's source code and its running version, then connects the two sets of results, so each real vulnerability appears once, with the line of code that causes it and the proof that it can be exploited.

The setting. Imagine a company that runs an online shop as a web application. Its security team uses two kinds of scanners. A static scanner (SAST) reads the source code without running it. A dynamic scanner (DAST) attacks the running application from the outside, the way a real attacker would. Each one produces its own report in its own format.

The problem. The two reports never meet:

- **Same bug, two tickets:** SAST says "SQL injection in orders.py, line 88". DAST says "SQL injection at /api/orders, parameter id". It is one bug, but nobody can see that, so it gets fixed twice or not at all.
- **Too much noise:** SAST reports hundreds of findings, many of them safe or in code that can never be reached. Developers stop trusting the list.
- **No place to fix:** a DAST finding proves the app can be attacked but does not say which code to change.
- **No clear priority:** a bug found in the code and proven exploitable matters more than a guess, but no report shows the difference.

Your challenge. Build a tool that scans source code (SAST), scans a running application (DAST) and correlates the two. You can build one combined tool, or separate SAST and DAST scanners that feed a shared correlation engine. Correlation is what we judge most: it must link a runtime finding to the code that causes it, merge duplicates and produce one prioritised list. You can choose any approach and any tech stack, and you may build on open-source scanners as long as you say so in the README.

Flagship Challenge. Any team that solves this problem earns bonus points on top of its normal score.

2. Goals and non-goals

A good solution finds real vulnerabilities in both the code and the running app, links the two views of the same bug, and gives developers one short list of what to fix first.

Goals

- **Scan code (SAST):** find vulnerabilities in the source code, with file and line.
- **Scan the running app (DAST):** find vulnerabilities by sending test requests, with the URL, parameter and payload.
- **One common format:** both scanners report findings in the same shape, using a CWE number for the vulnerability type.
- **Correlate:** link the code finding and the runtime finding of the same bug into one issue.
- **Prioritise:** show confirmed issues first, with a risk score.

- **Actionable:** for each issue, show where it is (URL and code line), how it was proven and what to fix.
- **Accurate:** few wrong links and few false alarms.
- **Safe:** scans only the test app that the organisers provide.

Not required

- A polished UI (a simple web page, a command-line report or a notebook is fine).
- Finding every type of vulnerability. A few types done well beat many done badly.
- Fixing the code automatically.
- Login, user management or cloud deployment.
- Support for many programming languages. The test app uses one.

What you will learn: static code analysis, web vulnerability testing, CWE and OWASP categories, matching findings across tools, and how to measure detection accuracy.

3. Functional requirements

Your tool must scan the test app's code and running instance, correlate the findings and show one prioritised list; the bonus items earn extra points. Cover at least four of the six vulnerability types in section 4, including SQL injection and cross-site scripting.

Must have

ID	Feature	Example
FR-1	SAST scan: scan a source folder and report findings with file, line, vulnerability type, severity and the code snippet	SQL injection, orders.py line 88, CWE-89
FR-2	DAST scan: scan a running app URL by sending test requests and report findings with URL, method, parameter, payload and evidence	GET /api/orders?id=1' returns a database error
FR-3	One common finding format for both scanners, with a CWE number for the type	Both sides report SQL injection as CWE-89
FR-4	Route map: read the source code to list which URL is handled by which function and lines	GET /api/orders → get_orders(), orders.py lines 80 to 95
FR-5	Correlate: link a DAST finding to a SAST finding when they have the same CWE and the DAST URL is handled by the code that holds the SAST finding	DAST SQL injection on /api/orders + SAST SQL injection at orders.py:88 = one confirmed issue
FR-6	Merge duplicates: one issue per real bug with all its evidence attached; running a scan twice does not create copies	3 scanner findings on the same bug = 1 issue
FR-7	Label every issue as Confirmed (found by both), Code only (SAST, not proven) or Runtime only (DAST, no code location found)	Hard-coded secret = Code only
FR-8	Issue detail for confirmed issues: URL, parameter, payload, file, line, code snippet, CWE and severity	One page that tells a developer where to look
FR-9	Prioritised list: confirmed issues first, then by risk score (severity and how easy it is to exploit)	Confirmed SQL injection above a Code only weak hash

ID	Feature	Example
FR-10	Simple command line, API or page, with a short README	POST /scans, GET /issues, GET /issues/7

Bonus (pick any)

ID	Feature
B-1	Auto-verify: for a Code only finding, send a safe test request to try to prove it at runtime and upgrade it to Confirmed
B-2	Cut SAST noise: mark findings as safe when the code is protected (e.g. a parameterised query) or no URL can reach it
B-3	Explain every link: show why two findings were joined (same CWE, same route, same parameter)
B-4	Plain-English fix suggestion for each confirmed issue
B-5	Compare two scans and show new, fixed and still-open issues
B-6	Export results as SARIF and fail a CI build on confirmed high-severity issues
B-7	Support a second programming language or framework

4. Test application, performance targets and setup

Organisers provide the test app and its answer key; hit the Good column to qualify and the Great column to stand out.

Test application (provided at kickoff)

- **One test app:** a small web shop with about 30 URLs, its source code and a Docker Compose file to run it on your own machine.
- **Planted vulnerabilities with an answer key:** 36 in total. 20 can be found in both the code and the running app (these are the pairs to link), 10 only in the code (e.g. hard-coded secrets) and 6 only at runtime (e.g. missing security headers).
- **Safe decoys:** about 8 pieces of code that look dangerous but are safe, such as a properly parameterised database query. Reporting one is a false alarm.
- **An unseen second app:** at judging, your tool also runs on a second app with the same kinds of bugs, to check that it is not tuned only to the test app.

Vulnerability types (cover at least four)

Type	CWE	Can be found by
SQL injection	CWE-89	Code and runtime
Cross-site scripting (XSS)	CWE-79	Code and runtime
Command injection	CWE-78	Code and runtime
Path traversal	CWE-22	Code and runtime
Open redirect	CWE-601	Code and runtime
Hard-coded secrets	CWE-798	Code only

Targets (measured against the answer key)

What we measure	How	Good	Great
Code detection	Share of planted code vulnerabilities your SAST finds	Over 60%	Over 85%
Runtime detection	Share of planted runtime vulnerabilities your DAST finds	Over 50%	Over 80%
Correct links	Share of the 20 planted pairs linked into one Confirmed issue	Over 50%	Over 80%
Wrong links	Share of your Confirmed issues that join two different bugs	Under 20%	Under 5%
False alarms	Share of reported issues that are safe decoys	Under 30%	Under 10%
Unseen app	Share of planted pairs linked on the second app	Over 40%	Over 65%
Run time	Time to scan, correlate and produce the report on the test app	Under 15 minutes	Under 5 minutes

Rules

- **Scan only the test app.** Point the DAST scanner only at the organisers' app running on your own machine. Scanning any other site or system is not allowed and leads to disqualification.
- **Say what you used.** Open-source scanners and rule sets are allowed, but list them in the README. The correlation logic must be your own work.

Suggested tools (not mandatory): Semgrep or Bandit for code scanning; OWASP ZAP or Nuclei for runtime scanning; Python's ast module or tree-sitter for the route map; SARIF as a common results format; SQLite or PostgreSQL to store findings.

Must run easily: judges should be able to start it by following the README (Docker Compose is a plus).

NOTE : **Bonus feature: reachability-aware SCA**

Report whether the app can actually reach a vulnerable library function, not just that a CVE exists.

1. **Inventory:** Parse lockfiles into an SBOM, keeping transitive paths.
2. **GHSA match:** Pull affected versions, fixes, CVSS, EPSS and KEV.
3. **Function map:** Build it from OSV symbols and fix-commit diffs, since GHSA rarely lists functions.
4. **Call graph:** Link entry points, app functions and library functions (CodeQL, Joern, Semgrep).
5. **Supply chain:** Check for malicious packages, typosquats, dependency confusion and unpinned lockfiles.

Statuses: Reachable, Imported-only, Present-only, Unknown (never downgrade Unknown).

Which language should this target first?